Agentic Credit Risk Intelligence Engine

Milestone 2 Project Report

Retrieval-Augmented Generation, LangGraph Orchestration, and LLM-Powered Risk Reporting

Team PowerPuff Boys

April 2026

Contents

1 Abstract . . . 2
2 Introduction . . . 2
3 Phase 1 Recap — The ML Foundation . . . 3
4 Phase 2 Overview — The Agentic Layer . . . 3
4.1 State Schema . . . 4
5 Node 1 — Risk Analyzer . . . 5
5.1 Preprocessing Pipeline . . . 5
5.2 Engineered Features . . . 5
5.3 Model Inference . . . 5
6 Node 2 — Guideline Retriever (RAG) . . . 6
6.1 Query Construction . . . 6
6.2 Embedding and Retrieval . . . 7
6.3 Output Format . . . 7
7 Node 3 — Report Generator . . . 7
7.1 Prompt Design . . . 7
7.2 Output JSON Schema . . . 8
7.3 LLM Configuration . . . 8
8 Knowledge Base Construction . . . 9
8.1 Source Documents . . . 9
8.2 Chunking Strategy . . . 9
8.3 Embedding and Storage . . . 10
9 LightGBM Upgrade from Phase 1 . . . 10
9.1 Why SMOTE . . . 10
9.2 Model Comparison . . . 10
10 Deployment — Streamlit Phase 2 UI . . . 11
10.1 Interface Structure . . . 11
10.2 Key Technical Decisions . . . 12
11 Tools and Technologies . . . 12
12 Results . . . 12
12.1 Model Performance . . . 13
12.2 Pipeline Performance . . . 13
12.3 Knowledge Base Coverage . . . 13
13 Conclusion . . . 13
13.1 Key Achievements . . . 14
13.2 Design Principles . . . 14
13.3 Future Directions . . . 14

1. Abstract

This report presents the second milestone of the Intelligent Credit Risk Scoring Engine — an agentic AI system built on top of the Phase 1 machine learning pipeline. Phase 2 extends the system from a static risk classifier into a fully autonomous, reasoning-capable credit risk assistant.

The architecture connects three sequential AI agents orchestrated by LangGraph: a LightGBM+SMOTE classifier that assigns a risk tier, a Retrieval-Augmented Generation (RAG) module that fetches relevant regulatory guidelines from a curated knowledge base, and a Groq-hosted large language model that synthesizes all signals into a structured, human-readable risk assessment report.

The Phase 1 model was upgraded from HistGradientBoosting to **LightGBM with SMOTE oversampling**, achieving **80.41% accuracy** and a Macro F1 of **0.71** across **100 features** on a 10,268-sample test set. The RAG knowledge base contains **2,931 document chunks** drawn from three official regulatory documents totalling 514 pages. The final system delivers end-to-end risk intelligence — from raw bureau data to a grounded, explainable JSON report — in approximately **10 seconds**.

2. Introduction

Credit risk classification alone answers only one question: *How risky is this borrower?* What it does not answer is equally important: *Why? What should the bank do? What does regulation say?*

Phase 1 of this project built a strong behavioral classifier using classical machine learning. It accurately predicted borrower risk tiers from CIBIL bureau data, excluding the Credit Score to ensure genuine behavioral learning rather than score replication.

Phase 2 goes further. It transforms the system into an **agentic AI pipeline** — one that not only classifies risk but also retrieves relevant lending guidelines, reasons over them, and produces a structured advisory report grounded in real regulatory documents. The system is designed to support credit officers with explainable, document-backed decision intelligence rather than a simple label.

The three core capabilities added in Phase 2 are:

- **Agentic orchestration** via LangGraph, connecting ML inference and AI reasoning in a stateful, fault-tolerant workflow

- **Retrieval-Augmented Generation (RAG)** using ChromaDB and semantic embeddings to fetch relevant regulatory context
- **LLM-powered report generation** via Groq (`llama-3.3-70b-versatile`) with strict JSON schema enforcement

3. Phase 1 Recap — The ML Foundation

Phase 1 delivered a behavioral credit risk classifier trained on 51,336 records merged from an internal bank dataset and external CIBIL bureau data. The key results are summarised below.

Aspect	Detail
Dataset size	51,336 records, 87 raw features
Target variable	`Approved_Flag` — four risk tiers (P1–P4)
Final model	HistGradientBoosting with cost-sensitive weighting
Accuracy	78%
Macro F1-Score	0.73
P3 (Medium) Recall	0.65
Deployment	Streamlit web interface with real-time prediction

Table 1: Phase 1 summary of key results.

The deliberate exclusion of `Credit_Score` was maintained throughout — the system learns from raw behavioral signals such as enquiry frequency, delinquency history, account age, and payment patterns rather than a derived bureau score. Phase 2 inherits this foundation entirely, reusing the preprocessing pipeline, feature engineering logic, and trained model artifact without modification.

4. Phase 2 Overview — The Agentic Layer

Phase 2 wraps the Phase 1 model inside a three-node LangGraph workflow. Each node has a single, well-defined responsibility. They execute sequentially, passing a shared typed state object from one to the next.

LangGraph Workflow — End-to-End Flow

Raw Bureau Features (from UI)

↓

Node 1: Risk Analyzer

LightGBM + SMOTE model · Risk tier, probabilities, top features

↓

Node 2: Guideline Retriever

ChromaDB + all-MiniLM-L6-v2 · 4 most relevant regulatory chunks

↓

Node 3: Report Generator

Groq llama-3.3-70b-versatile · Structured JSON risk assessment report

↓

Final Report (JSON) — displayed in Streamlit UI

The state object is defined as a typed Python dictionary (`TypedDict`) and carries all information between nodes. If any node encounters an error, it sets an error flag and subsequent nodes skip gracefully, preventing silent failures.

4.1 State Schema

Field	Type	Purpose
`prospect_id`	string	Unique identifier for the borrower
`features`	dict	Raw bureau features from the UI
`risk_tier`	string	P1 / P2 / P3 / P4 assigned by Node 1
`probabilities`	dict	Class probability scores for all four tiers
`top_risk_factors`	list	Top 5 features by model importance with values
`retrieved_chunks`	list	RAG results from ChromaDB
`report`	dict	Final structured JSON report from the LLM
`error`	string	Error message if any node fails

Table 2: LangGraph agent state schema.

5. Node 1 — Risk Analyzer

The Risk Analyzer is the entry point of the LangGraph workflow. It receives raw bureau features from the Streamlit UI and produces a complete risk classification.

5.1 Preprocessing Pipeline

The preprocessing logic exactly mirrors the training pipeline to prevent any train-serve skew. The steps are applied in the following order:

1. **Sentinel replacement** — Values of -99999 (used as missing indicators in the original dataset) are replaced with `NaN`.
2. **Column dropping** — Leakage and identifier columns (`Credit_Score`, `CC_utilization`, `PL_utilization`, `PROSPECT_ID`) are removed.
3. **Delinquency imputation** — Null values in six delinquency fields are filled with 0, representing no recorded delinquency event.
4. **Numeric imputation** — All remaining numeric nulls are filled with 0.
5. **Feature engineering** — Five composite behavioral features are computed (described in Section 5.2).
6. **Categorical encoding** — One-hot encoding with `drop_first=True` is applied to five categorical variables.
7. **Column alignment** — The dataframe is reindexed to exactly match the 100 training columns; any column missing from the input is filled with 0.

5.2 Engineered Features

Five composite behavioral features are computed at inference time, identical to those computed during training:

5.3 Model Inference

The node loads the frozen LightGBM model from `models/finalized_model.joblib` and the corresponding 100-column feature list from `models/feature_columns.joblib` — both serialised using `joblib`. Inference is performed through the `Booster.predict()` API, the lowest-level stable interface in LightGBM, which avoids operating-system-specific thread conflicts.

The node outputs three values into the shared state:

Feature	Formula	What It Captures
`delinq_burden`	$D_{\text{count}} \times D_{\text{level}}$	Severity-weighted delinquency load
`enq_pressure`	$E_{3m} \,/\, (E_{12m} + 1)$	Recency ratio of credit enquiries
`account_health`	$N_{\text{std}} \,/\, (N_{\text{sub}} + N_{\text{dbt}} + N_{\text{lss}} + 1)$	Ratio of standard to distressed accounts
`income_stability`	Income $\times$ $\ln(1 + T_{\text{employ}})$	Income weighted by employment tenure
`recency_risk`	$1 \,/\, (T_{\text{deliq}} + 1)$	Inverse decay of delinquency recency

Variables: D = delinquency counts/level, E = enquiry frequency (3m/12m), N = account status counts, T = time variables. All names match training pipeline features.

Table 3: Engineered behavioral features computed at inference.

- **Risk tier** (P1/P2/P3/P4) derived from `argmax` of class probabilities
- **Class probabilities** for all four tiers, retained as the authoritative confidence block
- **Top 5 risk factors** by feature importance with their actual input values, formatted as human-readable strings for the LLM prompt

6. Node 2 — Guideline Retriever (RAG)

The Guideline Retriever fetches regulatory context that is semantically relevant to the borrower's specific risk profile. It constructs a natural-language query from the risk tier and top risk factors, embeds it, and searches the ChromaDB vector store.

6.1 Query Construction

The query combines the risk tier and the top three risk factors into a single string. For example, a P3-tier borrower with high enquiry pressure and recent delinquency would produce a query similar to:

```
credit risk P3 borrower lending guidelines
enq_pressure = 0.43 (importance:  0.089)
time_since_recent_delinquency = 2 (importance:  0.071)
num_std_12mts = 3 (importance:  0.065)
```

6.2 Embedding and Retrieval

The retriever uses `all-MiniLM-L6-v2` from the SentenceTransformers library — a lightweight BERT-based model producing 384-dimensional sentence embeddings, optimised for semantic similarity tasks. The query is embedded locally with no external API call, then compared against stored document chunk embeddings using **cosine similarity**.

ChromaDB is configured with cosine distance (`hnsw:space: cosine`) and returns the **4 most relevant chunks** along with their source document names and similarity distance scores.

Implementation Note: ChromaDB Embedding Bug

ChromaDB 1.5.8's built-in `SentenceTransformerEmbeddingFunction` wrapper contains a threading bug that causes `collection.query(query_texts=...)` to hang indefinitely when invoked inside a LangGraph node. This was resolved by embedding the query directly with the raw `SentenceTransformer` model and using `query_embeddings=...` instead, bypassing the wrapper entirely. End-to-end pipeline time dropped from infinite to approximately 8–10 seconds after this fix.

6.3 Output Format

Each retrieved chunk is passed to Node 3 as a structured object containing:

- The chunk text (up to 500 characters)
- The source document name
- The cosine distance score (lower = more similar)

7. Node 3 — Report Generator

The Report Generator is the final node. It receives the risk classification, class probabilities, top risk factors, and retrieved regulatory chunks, then calls the Groq API to generate a structured, grounded risk assessment report.

7.1 Prompt Design

The system prompt establishes the LLM's role as a bank credit risk analyst and enforces four strict rules:

1. Only use facts from the provided guidelines — no invented statistics or regulations.
2. If guidelines do not cover a point, explicitly state "Insufficient guideline context."
3. Respond only with a valid JSON object — no markdown, no preamble.
4. All string values must be factual and concise.

The user message supplies: the prospect ID, risk tier, model probabilities, top risk factors, and the full text of all four retrieved guideline chunks.

7.2 Output JSON Schema

The LLM is required to respond in the following fixed schema:

Section	Field	Description
`summary`	`prospect_id`	Unique borrower identifier
	`risk_tier`	Assigned tier: P1, P2, P3, or P4
	`severity`	Human label: Very Low / Low / Medium / High
	`recommendation`	One-sentence lending action for the credit officer
	`overview`	2–3 sentence narrative of the borrower's risk profile
`deep_dive`	`top_risk_factors`	Top 3 features driving the risk tier
	`factor_explanations`	Plain-language explanation for each factor
	`guideline_references`	Direct quotes or paraphrases from retrieved regulations
	`suggested_actions`	Three concrete actions recommended for the credit team
`confidence`	`P1, P2, P3, P4`	Class probabilities from the LightGBM model (not the LLM)

Table 4: Output JSON report schema produced by Node 3.

After the LLM response is parsed, the `confidence` block is **overwritten** with the ML model's actual probabilities — the LLM's confidence values are discarded entirely. Probability scores must come from the calibrated LightGBM model, not from an LLM's self-reported certainty.

7.3 LLM Configuration

Parameter	Value	Reason
Model	`llama-3.3-70b-versatile`	Strong instruction-following, fast via Groq
Temperature	0.1	Near-deterministic output for consistent reports
Max tokens	1,200	Sufficient for the full JSON report schema
Provider	Groq API	Sub-second inference latency

Table 5: LLM configuration parameters.

8. Knowledge Base Construction

The RAG knowledge base is built offline by `ingest.py` and stored in a persistent ChromaDB directory (`chroma_db/`). It only needs to be rebuilt if the source documents change.

8.1 Source Documents

Three official regulatory documents were selected as the knowledge base:

Document	Source	Pages	Primary Coverage
RBI Master Circular — Basel III Capital Regulations	RBI	328	Capital adequacy, risk-weighted assets, credit exposure limits
IRACP Norms — Income Recognition, Asset Classification and Provisioning	RBI	77	NPA classification, provisioning requirements, restructuring guidelines
SEBI Master Circular — Credit Rating Agencies	SEBI	109	Rating methodology, disclosure obligations, monitoring requirements
Total		**514**	

Table 6: Regulatory documents in the RAG knowledge base.

8.2 Chunking Strategy

Each document is split into overlapping text chunks using a sliding window approach:

Overlap ensures that sentences spanning chunk boundaries are not lost, improving retrieval quality for regulatory phrases that fall near chunk edges.

Parameter	Value
Chunk size	500 characters
Overlap	100 characters
Minimum chunk length	50 characters (shorter chunks discarded)
Total chunks ingested	**2,931**

Table 7: Chunking parameters for the RAG knowledge base.

8.3 Embedding and Storage

All chunks are embedded in a single batch pass using `all-MiniLM-L6-v2` with `batch_size=64` and stored in ChromaDB with pre-computed embeddings. At query time, only the search query needs to be embedded — document embeddings are retrieved from disk. ChromaDB uses the HNSW (Hierarchical Navigable Small World) index with cosine distance for fast approximate nearest-neighbour search.

9. LightGBM Upgrade from Phase 1

Phase 1 used HistGradientBoosting as the final model. For Phase 2, the model was upgraded to **LightGBM with SMOTE oversampling**, achieving a measurable improvement in accuracy.

9.1 Why SMOTE

The dataset has a severe class imbalance: P2 (Low Risk) accounts for 62.7% of records, while P1, P3, and P4 together comprise only 37.3%. Phase 1 addressed this with cost-sensitive weighting. Phase 2 uses **SMOTE** (Synthetic Minority Oversampling Technique) to synthetically generate additional minority-class samples before training, creating a more balanced training distribution and enabling the model to learn stronger decision boundaries for underrepresented classes.

9.2 Model Comparison

LightGBM with SMOTE achieves the highest accuracy at **80.4%** with a Macro F1 of **0.71**, outperforming both the Decision Tree and Random Forest baselines. The improvement is driven by SMOTE's synthetic oversampling of minority classes and LightGBM's efficient gradient boosting over 500 estimators.

Model	Approach	Accuracy	Macro F1	P3 Recall
Decision Tree	Baseline (`max_depth=5`)	77.4%	0.67	0.30
Random Forest	GridSearchCV tuning	78.4%	0.71	0.41
LightGBM + SMOTE	**SMOTE + gradient boosting**	**80.4%**	**0.71**	**0.32**

Table 8: Model comparison across all experiments evaluated in the project.

10. Deployment — Streamlit Phase 2 UI

The Phase 2 interface (`agent_app.py`) is a complete rebuild of the Phase 1 Streamlit application, extended to expose the full agentic pipeline and its results.

10.1 Interface Structure

Sidebar

- Prospect selector — browse or randomise from the CIBIL unseen dataset
- Quick actions — Random prospect, Clear results buttons
- Team branding and model statistics

Risk Analysis Tab

- Prospect overview cards — gender, marital status, education, monthly income
- Bureau data inputs across three sub-tabs: enquiry and trade line data, delinquency and payment history, and demographics
- **Run Agentic Analysis** button — triggers the full LangGraph pipeline
- Risk tier card (color-coded: green for P1, amber for P2, orange for P3, red for P4)
- Risk overview and bank recommendation from the LLM
- Probability breakdown: four metric cards and animated bar charts for all tiers
- Deep dive panel: top risk factors with explanations, suggested actions, and guideline references
- Retrieved knowledge base chunks (expandable)
- Raw JSON report (expandable)

System Architecture Tab

- Annotated description of all three LangGraph nodes
- Full technology stack summary
- RAG knowledge base statistics

10.2 Key Technical Decisions

Lazy model loading — Heavy models (LightGBM, SentenceTransformers, ChromaDB) are loaded on first use rather than at startup, keeping the UI immediately responsive and only paying the initialisation cost when the user requests an analysis.

API key isolation — The Groq API key is loaded exclusively from a `.env` file using `python-dotenv` and is never exposed in the UI. If the key is missing, the application surfaces a clear error rather than failing silently.

Cached resource management — The CIBIL dataset is loaded once using `@st.cache_data` and the agent runner is initialised once using `@st.cache_resource`, preventing redundant disk reads and model loads across Streamlit reruns.

11. Tools and Technologies

Category	Tool	Purpose
Language	Python	Primary runtime
ML Model	LightGBM	Risk classification
Class Balancing	imbalanced-learn	SMOTE oversampling
Data Processing	pandas, NumPy	Feature engineering
Workflow Orchestration	LangGraph	Agentic node graph
LLM Provider	Groq API	Fast LLM inference
LLM Model	llama-3.3-70b-versatile	Report generation
Embeddings	sentence-transformers	Semantic query embedding
Embedding Model	all-MiniLM-L6-v2	384-dim sentence vectors
Vector Database	ChromaDB	Chunk storage and retrieval
PDF Ingestion	pypdf	Regulatory document parsing
UI	Streamlit	Interactive web interface
Model Persistence	joblib	Frozen model artifact loading
Environment Config	python-dotenv	Secure API key management
Version Control	Git, GitHub	Source control
Development	Jupyter Notebook	Experimentation and training

Table 9: Tools and technologies used in Phase 2.

12. Results

12.1 Model Performance

Metric	Value
Overall Accuracy	**80.41%**
Macro F1-Score	**0.71**
Weighted Precision	0.79
Weighted Recall	0.80
Test Set Size	10,268 samples
Training Features	100
Estimators	500
Balancing Strategy	SMOTE
Model Type	LightGBM (LGBMClassifier)

Table 10: Phase 2 LightGBM + SMOTE model performance on the test set.

12.2 Pipeline Performance

Stage	Description	Typical Duration
Node 1 — Risk Analyzer	Preprocessing + LightGBM inference	$<$ 1 second
Node 2 — Guideline Retriever	Query embedding + ChromaDB search	1–3 seconds
Node 3 — Report Generator	Groq API call + JSON parsing	4–7 seconds
Total end-to-end		**∼8–10 seconds**

Table 11: End-to-end pipeline timing breakdown.

12.3 Knowledge Base Coverage

Document	Chunks	Avg. Chunk Length
RBI Basel III Capital Regulations	≈1,870	∼480 chars
IRACP Norms	≈440	∼480 chars
SEBI Credit Rating Agencies Circular	≈621	∼480 chars
Total	**2,931**	

Table 12: Knowledge base chunk distribution by source document.

13. Conclusion

Phase 2 of the Intelligent Credit Risk Scoring Engine successfully transforms a static ML classifier into a fully agentic AI system capable of end-to-end credit risk intelligence.

13.1 Key Achievements

- **Agentic workflow** — A three-node LangGraph pipeline connects machine learning, vector search, and large language model reasoning in a stateful, fault-tolerant graph.
- **Model upgrade** — LightGBM + SMOTE achieves 80.4% accuracy, a 2.4-point improvement over Phase 1.
- **Grounded recommendations** — All LLM outputs are constrained to retrieved regulatory guidelines, preventing hallucination and ensuring compliance-safe advisory content.
- **Probability integrity** — Model probability scores are injected directly into the final report; the LLM's own confidence is always discarded.
- **2,931 regulatory chunks** — RBI and SEBI regulatory content covering capital adequacy, NPA classification, and credit rating obligations is available for retrieval.
- **Production hardening** — Apple Silicon segfault, ChromaDB hang, and tokenizer deadlock issues were all diagnosed and resolved, making the system portable across macOS environments.

13.2 Design Principles

The system was built around three principles that distinguish it from a simple LLM wrapper:

1. **The ML model decides risk.** The LLM has no influence over the risk tier or probability scores. It only explains and advises based on what the model outputs.
2. **The knowledge base constrains advice.** Recommendations are grounded in retrieved regulatory documents, not the LLM's parametric knowledge, making outputs auditable and traceable.
3. **Failures are explicit.** Every node propagates errors through the shared state object, making failures visible and debuggable in the UI rather than silently swallowed.

13.3 Future Directions

- **Feedback loop** — Capture analyst overrides to fine-tune the classifier on real-world credit decisions over time.
- **Multi-document reasoning** — Allow the LLM to synthesize relationships across all retrieved chunks rather than treating them as independent references.

- **Explainability layer** — Integrate SHAP value computation per prospect for regulatory-grade model explainability reports.
- **Streaming output** — Stream the LLM report token-by-token for a more responsive and interactive UI experience.

Report prepared by Team PowerPuff Boys — Agentic Credit Risk Classification, April 2026
LightGBM + SMOTE · LangGraph · ChromaDB · Groq llama-3.3-70b-versatile · Streamlit